coursera | UNIVERSITY OF LONDON

Mathematical Foundations for Computing

Module 1
Module 1: Sets and Set Theory

Module 2
Module 2: Number Systems and Bases

Lesson 2.1 Introduction to Number Bases

- Introduction to number bases
  Video • 13 min
- Introduction to number bases – conversion to decimal
  Practice Assignment • Grade: 100%
- Conversion of integer numbers between different bases
  Video • 7 min
- Conversion of non-integer numbers between bases
  Video • 14 min
- Non-integer numbers conversion
  Practice Assignment • Grade: 100%

Lesson 2.2 Operations with Binary Numbers

- Operations with binary numbers – addition
  Video • 3 min
- Operations with binary numbers – subtraction
  Video • 4 min
- Operations with binary numbers – multiplication
  Video • 4 min
- Number representations
  Video • 8 min
- Operations with binary numbers
  Practice Assignment • Grade: 100%
- **Why do we need to use these number conversions?**
  Dialogue • 15 min

Lesson 2.3 Summary and Assessments

- Topic 1 – looking back
  Video • 2 min
- Check your understanding: End of module 2
  Graded Assignment • 20 min

Module 3
Module 3: Functions and Their Properties

Module 4
Module 4: Relations

Lesson 4.1 Understanding the concept of relations

- Definition of a relation: relation versus function
  Video • 8 min
- Definition of a relation: relation versus function
  Practice Assignment • 20 min
- Matrix and graph representations of a relation
  Video • 10 min
- Matrix and graph representations of a relation
  Practice Assignment • 20 min
- The properties of a relation: reflexive, symmetric and anti-symmetric
  Video • 14 min
- The properties of a relation: reflexive, symmetric and anti-symmetric
  Practice Assignment • 20 min
- Relation properties: transitivity
  Video • 5 min
- Relation properties: transitivity
  Practice Assignment • 20 min

Lesson 4.2 Equivalence, and partial and total order relations

- Equivalence relations and equivalence classes
  Video • 6 min
- Equivalence relations and equivalence classes
  Practice Assignment • 25 min
- Partial and total order
  Video • 6 min
- Partial and total order
  Practice Assignment • 30 min
- Building and analysing different types of mathematical relations
  Dialogue • 15 min

Lesson 4.3 Summary and Assessments

- Topic summary
  Reading • 15 min
- Check your understanding: End of module 4
  Graded Assignment • 20 min

Course Wrap-Up

- Mathematical Foundations for Computing: Course Summary
  Reading • 10 min
- The Archive Room: A Mathematical Escape Mystery
  Role Play • 15 min
- Course summary
  Video • 1 min

Why do we need to use these number conversions?

Welcome! In this Dialogue, we'll explore why number base conversions are essential in computer science by connecting specific computing applications to the mathematical concepts you've learned.

Here's what we'll cover:

- **Computing Hardware Foundations:** We'll explore how binary representation connects to the fundamental operation of digital computers and electronic circuits.
- **Data Storage and Memory:** We'll examine how different number bases optimize data storage, memory addressing, and file compression.
- **Programming and Algorithms:** We'll analyze practical applications in programming languages, debugging, and algorithm efficiency.

Need help? Click "I'm stuck" at the top right to get a hint.

When you're ready, click "Start Dialogue"

Great! Let's dive into our first topic: **Computing Hardware Foundations**. How do you think the binary number system is fundamental to the way digital computers and electronic circuits operate?

S **Sandipan**

The digital computers can represent 0 and 1 by two stable states of a transistor OFF and ON, hence operate in binary number sustem

That's a great start! You've correctly identified that the ON and OFF states of a transistor directly correspond to 1 and 0 in the binary system. This is indeed the fundamental way computers represent information. Can you elaborate a bit more on *why* this binary representation is so crucial for the actual operation of these circuits? What advantages does using just two states offer for hardware?

S **Sandipan**

All the hardware circuits take input and output as ON / OFF states, with much higher accuracy than analogue system

You're absolutely right that digital systems, by using ON/OFF states, offer much higher accuracy than analog systems. This stability is a huge advantage!

Now, let's move on to **Data Storage and Memory**. How do different number bases, beyond just binary, play a role in optimizing data storage, memory addressing, and even file compression?

S **Sandipan**

Higher bases allow more compact representation with less storage

That's a key insight! Using higher bases like hexadecimal can indeed represent large binary numbers more compactly, which is very useful for human readability and managing memory addresses. Can you give an example of how this compact representation might be beneficial when dealing with computer memory or data storage, perhaps thinking about how a programmer might view memory locations?

S **Sandipan**

With 32 bit or 64 bit addresses it's much easier to interpret and address / data with hex presentation requiring only 8 or 16 hex digits, more managable

Excellent point! You've highlighted exactly why hexadecimal is so practical for memory addresses. Trying to read and write 32 or 64 binary digits would be incredibly cumbersome and error-prone. Hexadecimal provides a much more manageable and human-friendly way to represent those long binary strings.

Now, let's move on to our final topic: **Programming and Algorithms**. How do number base conversions come into play when you're writing code, debugging, or thinking about the efficiency of an algorithm?

S **Sandipan**

For the algorithm complexity / runtime / performance testing powers of 2 become very important.

You're on the right track! Powers of 2 are indeed very important when analyzing algorithm complexity, especially in areas like data structures (think binary trees) or algorithms that divide problems in half. Can you give a more specific example of how understanding number bases, or perhaps even performing a conversion, might directly help a programmer when they are *writing* code, *debugging* an issue, or *optimizing* an algorithm?

S **Sandipan**

Having concepts about how the numbers are stored exactly in terms of binary or hexadecimal numbers may help programmer handle overflow, underflow, signed / unsigned numbers, floating point number and accuracy losses. Also, knowledge on how the numbers are stored around memory word boundaries can make the programmer to write more memory effiicient programs

That's a fantastic and comprehensive answer! You've hit on several critical points: understanding how numbers are stored (binary/hexadecimal) is essential for handling issues like overflow, underflow, and the nuances of signed/unsigned and floating-point numbers, which directly impacts accuracy. Furthermore, recognizing memory word boundaries for efficient memory usage is a hallmark of an expert programmer.

Good job, you have completed all the topics for our session. Up next - view your feedback.

During today's session covering why number base conversions are essential in computer science, we focused on **Computing Hardware Foundations**, **Data Storage and Memory**, and **Programming and Algorithms**.

Your strengths:

- You demonstrated a strong understanding of how binary representation underpins digital computer operations, specifically linking it to transistor states and the accuracy benefits over analog systems.
- You clearly articulated the practical advantages of higher number bases like hexadecimal for compact data representation and memory addressing, especially for human readability and managing large addresses.
- You provided excellent insights into how a deep understanding of number storage (binary/hexadecimal) is crucial for programmers to handle complex issues like overflow, underflow, signed/unsigned numbers, floating-point accuracy, and memory efficiency.

Areas for improvement:

- While you touched upon algorithm complexity, further exploration into how specific number base conversions (e.g., converting between binary and decimal for specific calculations) directly impact algorithm design or debugging scenarios could deepen your understanding.

Keep up the good work!

The Dialogue has ended. Start a new chat to clear your chat history and start fresh.

Try again ✓ Completed

Dialogue is powered by AI, so check for mistakes and don't share any personal/sensitive information.

Like Dislike Report an issue Tell us what you think

Go to next item →